

PROJETO

Codifique um pouso em Marte

Na NASA, o processo que chamamos de entrada, descida e pouso, ou EDL, é a série de eventos que ocorrem desde o momento em que uma espaçonave encontra o topo da atmosfera marciana até que ela toque com segurança na superfície. Você pode modelar esse processo usando linguagens de codificação, como Python! Nesta atividade, você irá programar vários recursos de EDL, como determinar a proximidade da sua nave espacial da superfície quando ela chegar a Marte.



Materiais

Dispositivo capaz de executar código Python

Software de edição Python grátis, como Mu ou Atom

Sensor de luz

LEDs de várias cores

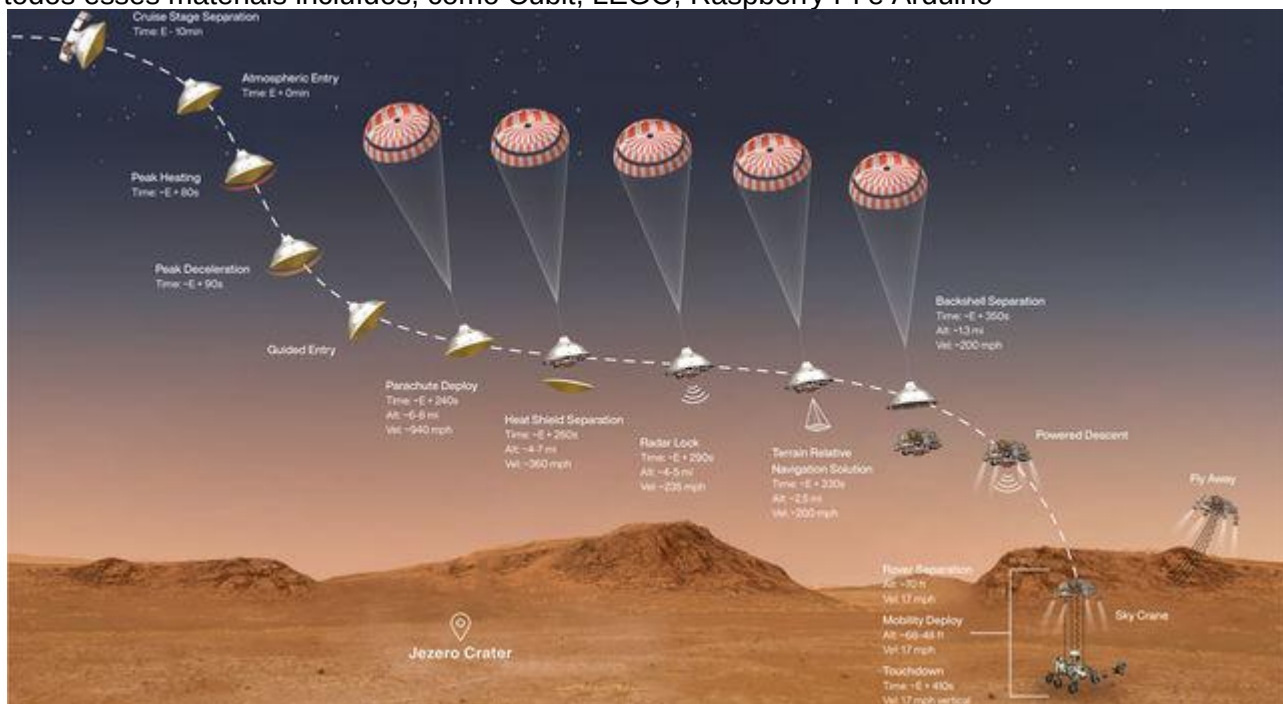
Botões programáveis

Sensor ultrassônico

Campainha ou alto-falante

Cabos conectores e placa de ensaio

Embora essa atividade possa ser concluída com o uso de componentes individuais, existem vários kits com todos esses materiais incluídos, como Cubit, LEGO, Raspberry Pi e Arduino



1. Aprenda o que é preciso para pousar em Marte

Aterrar uma espaçonave em Marte não é uma tarefa fácil. Por causa da enorme distância entre a Terra e Marte, não podemos controlar a espaçonave em tempo real como faria em um videogame. Todas as nossas comunicações com a espaçonave são limitadas pela velocidade da luz, então um sinal leva cerca de sete minutos para ser enviado da Terra a Marte e outros sete minutos para voltar. Como resultado, as espaçonaves precisam pousar de forma autônoma - por conta própria, com base nas instruções programadas em seus computadores de bordo.

Para pousar com sucesso uma espaçonave, nós a enviamos para o espaço com centenas de milhares de linhas de código, fornecendo instruções para cada uma das manobras que ela precisará realizar para pousar com segurança, incluindo:

- Medir a que distância está da superfície
- Saber quando realizar cada manobra, como lançar seu paraquedas
- Comunicando seu status aos controladores de missão na Terra
- E iniciar as operações com segurança ao chegar à superfície

Sobre a imagem: Esta ilustração mostra os eventos que ocorrem nos minutos finais da jornada de quase sete meses que o rover Perseverance da NASA leva a Marte. Crédito da imagem: NASA / JPL-Caltech |

2. Prepare-se

Reveja cada uma das tarefas que você precisará concluir nas etapas abaixo para pousar sua nave espacial com segurança. E obtenha mais inspiração pesquisando [como a NASA pousa a espaçonave em Marte](#). Em seguida, decida quais dispositivos você usará para simular sua espaçonave e o pouso. Quais ferramentas você tem para indicar as etapas bem-sucedidas ao longo do processo de desembarque? Você usaria luzes em certas etapas? As luzes devem estar acesas ou apagadas? Você poderia usar som? Alguns dispositivos são capazes de detectar cores - talvez a cor do local de pouso marciano? Seja tão criativo quanto quiser e torne-o seu com base em suas próprias ideias, no que você tem disponível e no que você se sente confortável. Observação: ao longo desta atividade, forneceremos alguns códigos e nomes de amostra, mas os seus provavelmente irão variar dependendo do que você está usando. Apenas lembre-se de ser consistente o tempo todo!

Tarefas

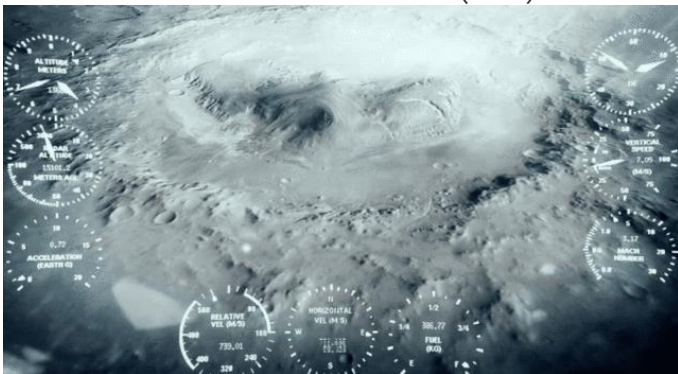
1. **Importe seus componentes** - alguns dispositivos fazem isso automaticamente, enquanto outros usam uma linha de 'importação' para chamar em sua biblioteca. Cada dispositivo tem suas próprias convenções de nomenclatura e vocabulário.

```
de importação de componentes UltrasonicSensor, LED
```

```
do tempo importar dormir
```

```
#Se o seu kit não detectar automaticamente a porta que está sendo usada, você pode especificar
```

```
ultrasonic = UltrasonicSensor ("D1")
```



3. Meça a distância entre a espaçonave e a superfície

Como o nome sugere, a entrada, a descida e a aterrissagem são um processo de várias etapas. Por exemplo, os engenheiros programam o rover para fornecer feedback constante sobre sua altimetria - altura - enquanto desce em Marte. Uma vez que uma espaçonave determina que está em uma determinada altitude, ela sabe como iniciar a próxima fase da sequência de pouso.

Tarefas

1. **Determine seu local de pouso** - você determinará sua “meta de pouso” com base nos dispositivos que está usando para simular a aterrissagem. Por exemplo, se você tiver um pequeno dispositivo para usar como sua nave espacial, poderá amarrá-lo a um barbante e baixá-lo da borda de uma mesa em direção ao chão. Se o seu dispositivo for maior ou delicado, você pode tentar deslizar o dispositivo horizontalmente pelo chão em direção a uma parede ou um alvo.
2. **Configure um sensor ultrassônico** - comece escrevendo o código para ativar o sensor ultrassônico e fazer com que ele exiba a distância que está lendo.

```
enquanto verdadeiro:
    distance = ultrasonic.distance
    imprimir (distância)
    dormir (0,1)
```

3. **Adicione um indicador de luz para refletir a medição** - Usando LEDs, podemos criar rapidamente uma saída visual para refletir a medição do sensor ultrassônico. Isso pode ser feito usando um padrão ir-cautela-parar, mas sinta-se à vontade para personalizá-lo!

```
red_led = LED ("D2")
yellow_led = LED ("D3")
green_led = LED ("D4")
se distância <10:
    red_led.on ()
    yellow_led.on ()
    green_led.on ()
distância elif <20:
    red_led.off ()
    yellow_led.on ()
    green_led.on ()
outro:
    red_led.off ()
    yellow_led.off ()
    green_led.on ()
```

4. Crie um alarme para cada fase do pouso

O rover Perseverance tem uma câmera abaixo dele que pode medir a distância entre ele e a superfície marciana. Isso permite que a espaçonave determine quando realizar cada uma de suas manobras de pouso, como disparar retrofoguetes para diminuir ainda mais a descida da espaçonave.

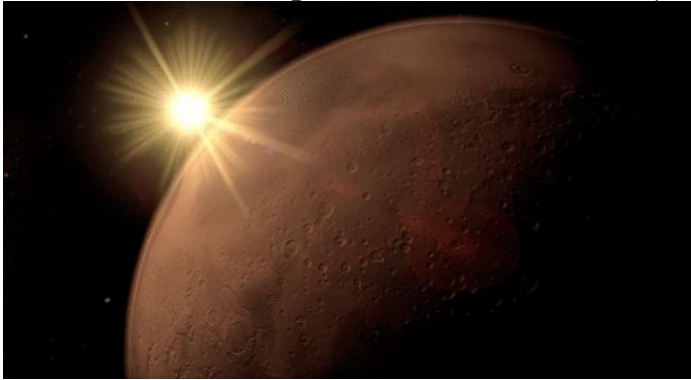
Tarefas

1. **Transição entre estágios** - adicione um sistema de alarme que sinaliza quando é hora de passar para a próxima fase de pouso da sua espaçonave, como lançar os cabos do skycrane que abaixam o rover até o solo. Um exemplo de alerta de áudio é fornecido abaixo.

```
de componentes importar Buzzer
campanha = campanha ("D5")
se distância <10:
    red_led.on ()
    yellow_led.on ()
    green_led.on ()
    buzzer.on ()
distância elif <20:
    red_led.off ()
    yellow_led.on ()
    green_led.on ()
outro:
    red_led.off ()
    yellow_led.off ()
    green_led.on ()
```

2. **Considere como ajustar para a altura desejada** - O código de amostra fornecido acima soa o alarme quando nosso código atinge uma certa leitura de luz e distância. Você pode modificar o código para produzir uma luz verde para implantar na distância desejada - nem muito perto nem muito longe?

Sobre a imagem: Uma cena animada do trailer da missão para o pouso do Perseverance Mars rover. Crédito da imagem: NASA / JPL-Caltech |



5. Ligue

Enquanto os rovers Perseverance e Curiosity são [movidos pelo que é conhecido como RTG](#), outras missões a Marte utilizam energia solar, coletando luz através de painéis solares e convertendo-a em eletricidade. Uma vez que essas missões pousaram com segurança, eles começam a coletar luz para ligar e começar a se comunicar com os cientistas na Terra.

Tarefas

1. **Colete a luz solar para alimentar sua espaçonave** - Usando um sensor de luz ou painel solar, podemos medir a quantidade de luz solar que estamos coletando para fornecer energia para nossa espaçonave.

```
de componentes importar LightSensor
do tempo importar sono, tempo
luz = LightSensor ("D6")
enquanto verdadeiro:
    lightlevel = (light.reading)
    imprimir nível de luz
```

2. [ver crucedl step5-1](#) hospedado com ❤ pelo [GitHub](#)

3. **Plote seus dados de energia solar** - Dependendo do seu dispositivo ou kit, pode ser possível imprimir uma leitura constante de quanta luz sua nave está coletando ou até mesmo plotar os dados ao longo do tempo. A capacidade de ver isso em tempo real irá variar dependendo se a tela está no kit ou no computador.

```
plt.ion ()
fig = plt.figure ()
x_data = []
y_data = []
start_time = time ()
x_data.append (elapsed_time)
y_data.append (lightlevel)
plt.title ("Entrada do sensor de luz ao longo do tempo")
plt.ylabel ("Nível de luz")
plt.xlabel ("Tempo em segundos")
plt.plot (x_data, y_data)
plt.draw ()
plt.pause (0,01)
```

6. Telefone para casa

Lembre-se de que, devido à vasta distância entre a Terra e Marte, não sabemos realmente se a missão pousou com sucesso até que tenha transmitido seu status de volta para nós.

Tarefas

1. **Envie uma mensagem usando som** - Programe uma campainha para enviar manualmente uma mensagem de volta à Terra, sinalizando que sua nave está segura e ligada. Experimente fazer com que a campainha soe uma mensagem simples em código Morse. Um exemplo de dicionário de código Morse pode ser encontrado no trecho de código abaixo.

```
do botão de importação de componentes
botão = Botão ("D7")
enquanto verdadeiro:
    if button.is_pressed:
        buzzer.on ()
    outro:
        buzzer.off ()
```

2. **Desafio: Envie sua mensagem em código, em vez de som** - as ondas sonoras não podem viajar pelo vazio do espaço, portanto, não podemos usar o som para que a espaçonave transmita seu status seguro de volta à Terra. Em vez disso, podemos fazer com que a espaçonave escreva sua mensagem em código Morse e, em seguida, transmita essa mensagem de volta para nós usando luz.

```
Morse_Dict = {'A': '.-.', 'B': '-...', 'C': '-.-.', 'D': '- ..', 'E': '. .',
'F': '...-', 'G': '-.-', 'H': '....', 'I': '..-', 'J': '.... ', 'K': '-.- ',
'L': '.- ..', 'M': '--', 'N': '-.', 'O': '---', 'P': '---.', 'Q': '---.',
'R': '---.', 'S': '...', 'T': '--', 'U': '...-', 'V': '....-', 'C': '---',
'X': '---.', 'Y': '-.-.-', 'Z': '- ..'}
def encrypt (mensagem):
    convert = ''
    para carta em mensagem:
        se letra! = '':
            convert + = Morse_Dict [letra] + ' '
        outro:
            convert + = ' '
    retorno convertido
def decrypt (mensagem):
    mensagem + = ' '
    decodificar = ''
    texto = ''
    para carta em mensagem:
        if (letra! = ' '):
            i = 0
            texto + = letra
        outro:
            i + = 1
            se i == 2:
                decodificar + = ' '
            outro:
                decodificar + = list (Morse_Dict.keys ()) [list (Morse_Dict.values ()) . index (texto)]
                texto = ''
    decodificação de retorno
def main ():
    text = input ("Digite a mensagem:")
    output = encrypt (text.upper ())
    imprimir (saída)
if __name__ == '__main__':
    a Principal()
```

Isso é semelhante a como nos comunicamos com a espaçonave por meio [da Deep Space Network da NASA](#) , ou DSN. Os dados são convertidos em código binário (uns e zeros) por computadores e transmitidos como ondas de luz de espaçonaves para as gigantescas antenas parabólicas que compõem o DSN.

Sobre a imagem: Uma cena animada de Marte em um vídeo de minuto "Phoning Home - Communicating From Mars." Crédito da imagem: NASA / JPL-Caltech



7. Reúna tudo

Como aprendemos, devido à distância entre a Terra e Marte, nosso código deve ser executado de forma autônoma. Assim, uma vez que o EDL começa, nosso código precisa ser executado por conta própria do início ao fim.

Tarefas

Coloque seu código em uma sequência - como um desafio final, veja se você pode editar seu código para fazer todas as ações a seguir em sequência:

1. Meça a distância da superfície, passando de um LED vermelho (muito alto para implantar) para um LED amarelo para um LED verde (altura desejada para implantar).
2. Quando sua espaçonave atingir a altura desejada da superfície (luz verde), faça com que ela toque um som.
3. Ao reproduzir o som indicando que ele pousou com segurança, comece a coletar a luz.
4. Após 30 segundos de coleta de luz, solicite uma mensagem em código Morse para transmitir de volta à Terra.
5. Assim que a saída do código Morse for recebida (manualmente ou pré-programada), faça com que seu programa saia das operações (desligue os LEDs e pare de coletar luz).

Sobre a imagem: Uma ilustração do rover Perseverance da NASA pousando com segurança em Marte. Crédito da imagem: NASA / JPL-Caltech |